Java 21

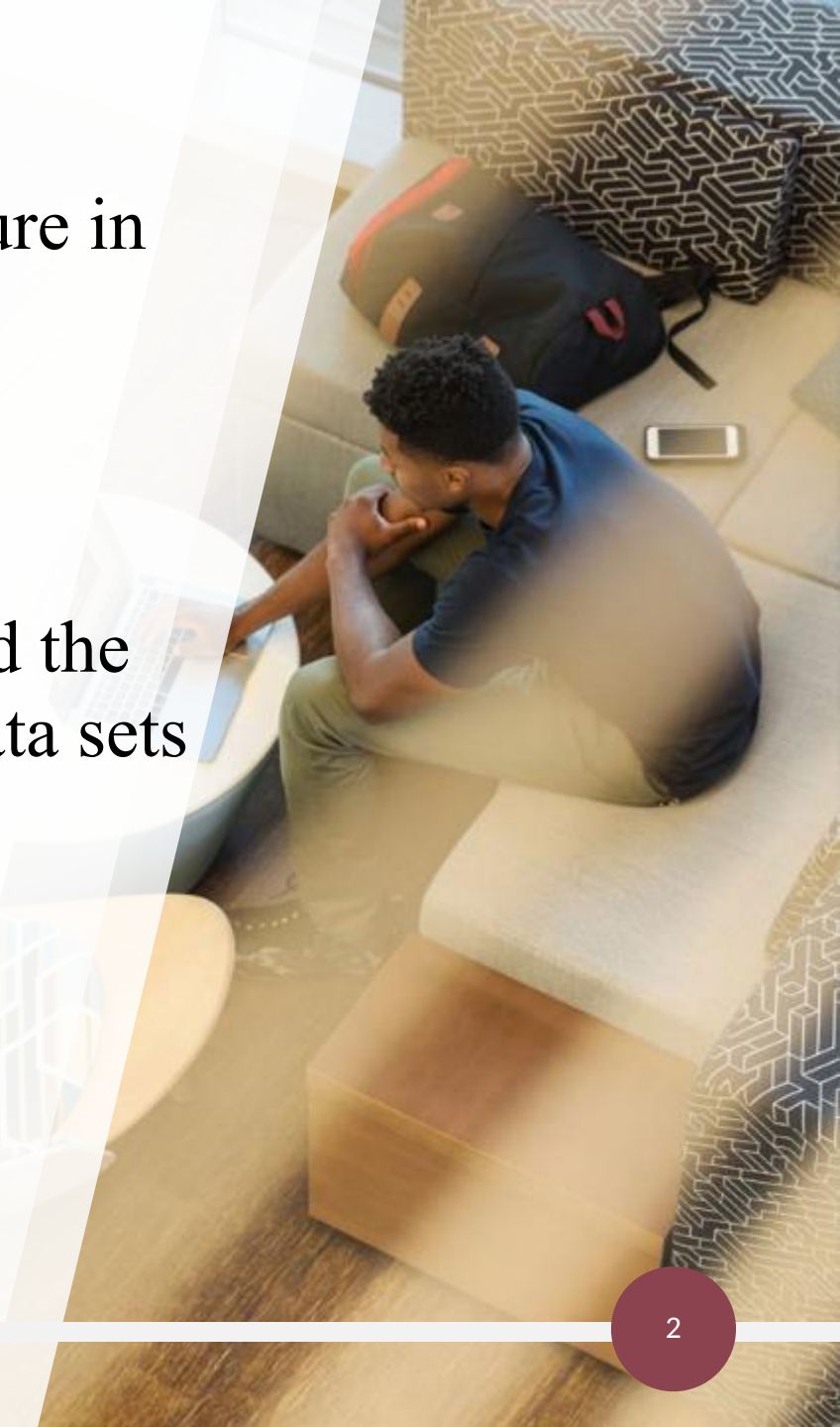
Virtual Threads

Virtual Threads - Background

- Virtual threads were first introduced as a preview feature in Java 19.

- Now, Java 21 finalizes the feature.

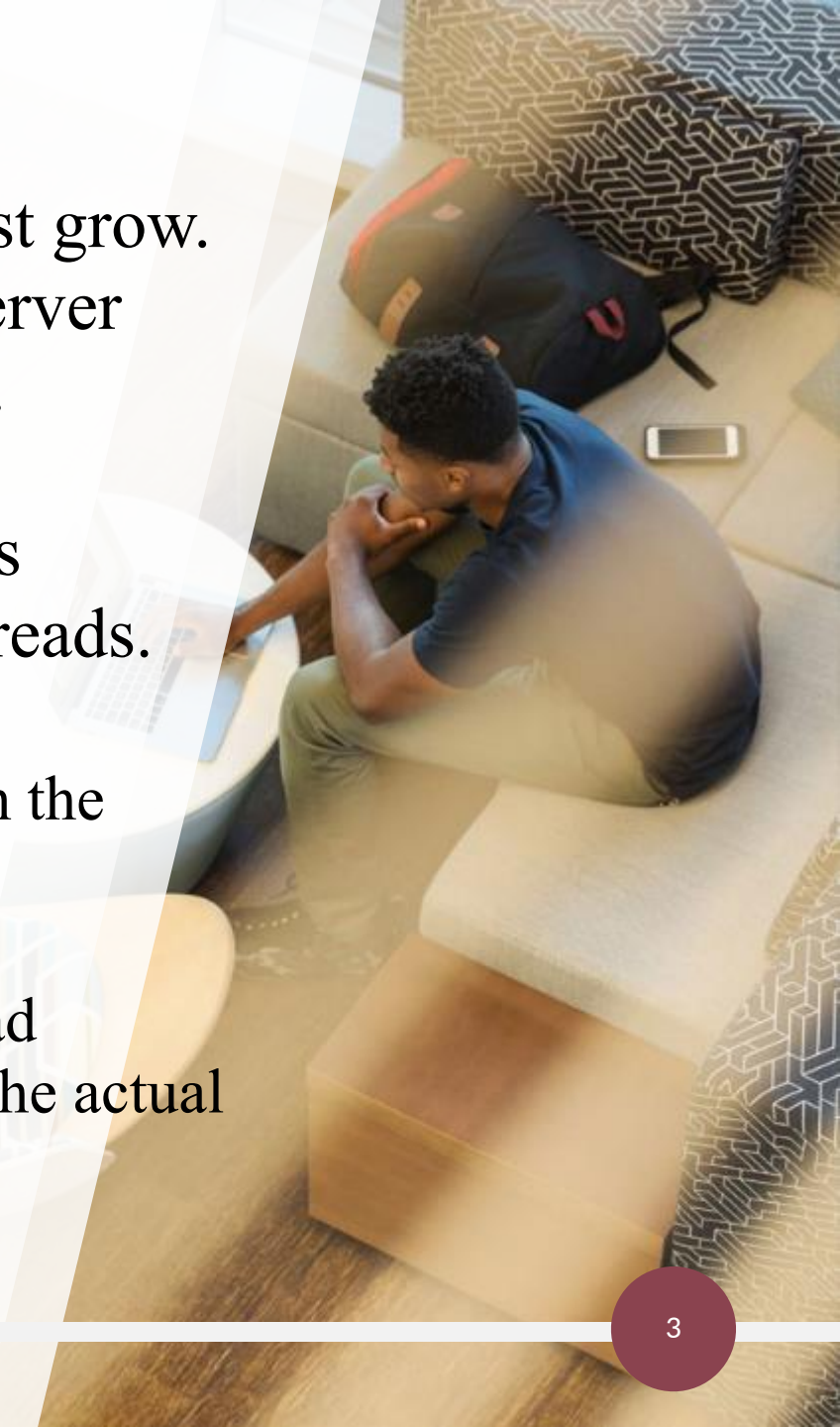
- The basic concurrency model of Java is unchanged and the Stream API is still the preferred way to process large data sets in parallel.



Virtual Threads - Motivation

- As a server application scales, the number of threads must grow. A main goal of virtual threads is to enable scalability of server applications written in the simple thread-per-request style.

- The JDK's current implementation of threads implements threads as thin wrappers around operating system (OS) threads. However, as OS threads are expensive:
  - if each request consumes an OS thread for its duration, then the number of threads often becomes the limiting factor when attempting to scale.
  - thus, an application's throughput is capped even when thread pooling is employed (given that pooling does not increase the actual number of threads).



Virtual Threads - The Goal

- The goal is to break this 1:1 relationship between Java threads and OS threads.

- Consider operating systems presenting the illusion of plentiful memory by mapping a large virtual address space to a limited amount of physical RAM.

- Similarly, the Java runtime can give the illusion of plentiful threads by mapping a large number of *virtual* threads to a small number of OS threads.

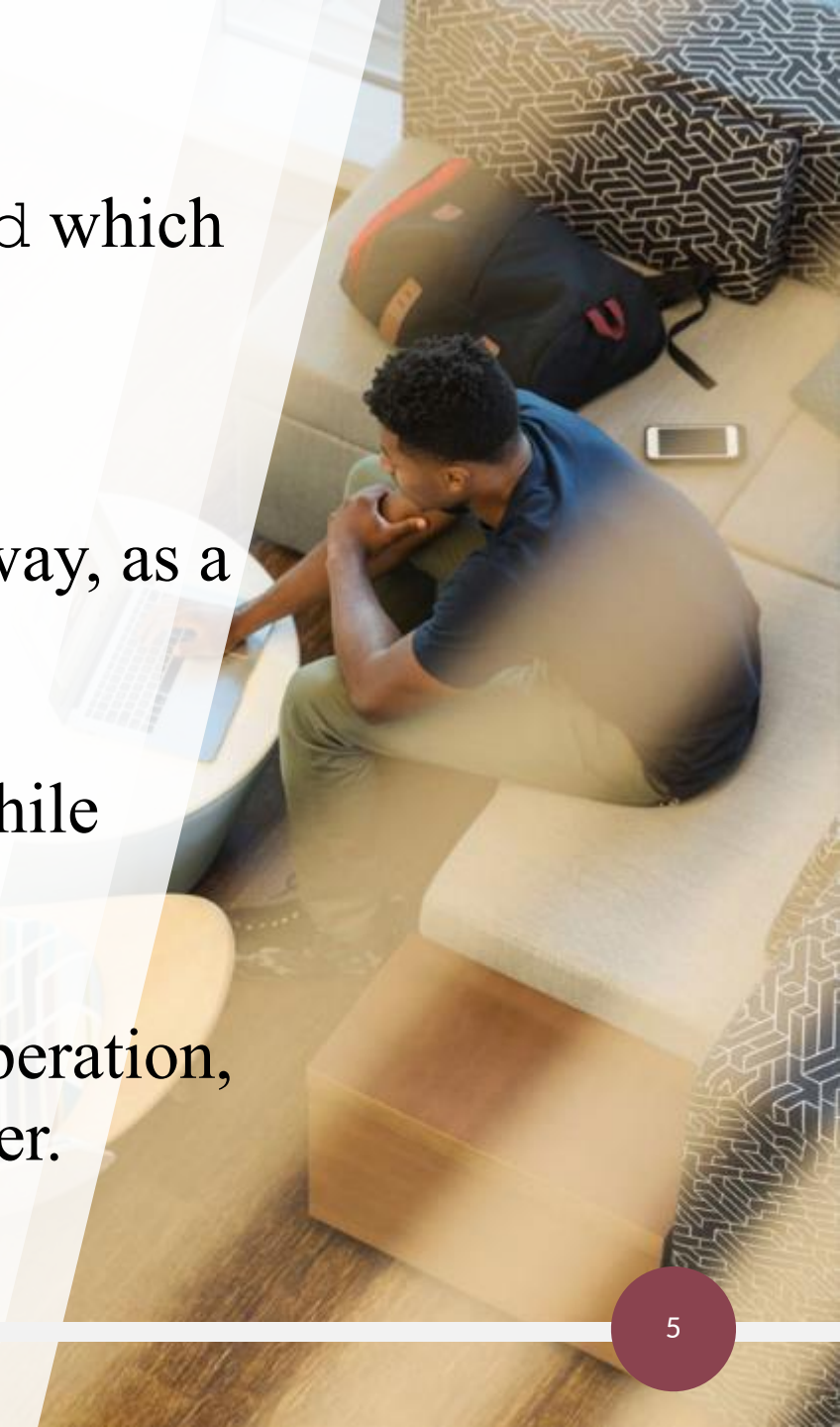
Virtual Threads versus Platform Threads

- A *virtual thread* is an instance of `java.lang.Thread` which is not tied to any particular OS thread.

- A *platform thread*, by contrast, is an instance of `java.lang.Thread` implemented in the traditional way, as a thin-wrapper around an OS thread.

- Importantly, virtual threads only consume OS threads while performing calculations **on the CPU**.

- In addition, when a virtual thread executes a blocking operation, it is **automatically suspended** until it can be resumed later.



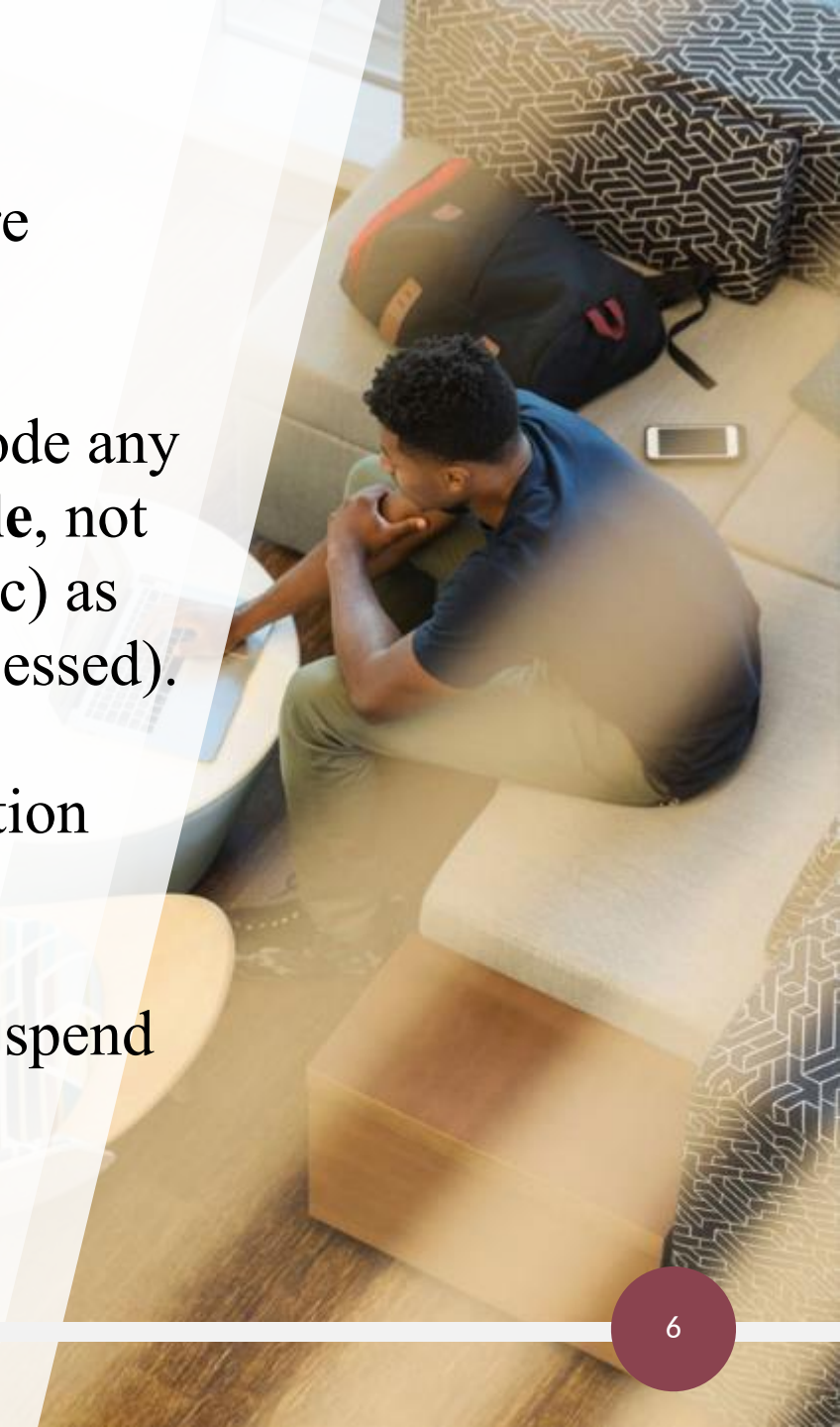
Virtual Threads - when to use

• From our perspective, virtual threads are simply threads that are cheap to create and hugely plentiful.

• Virtual threads are not faster threads, in that, they do not run code any faster than platform threads. Virtual threads exist to provide **scale**, not speed. In other words, higher throughput (number of requests/sec) as opposed to lower latency (length of time for a request to be processed).

• In a nutshell: Virtual threads can significantly improve application throughput when:

- number of concurrent tasks is high (> a few thousand).
- the workload is **not** CPU-bound - ideally the tasks need to spend much of their time waiting.

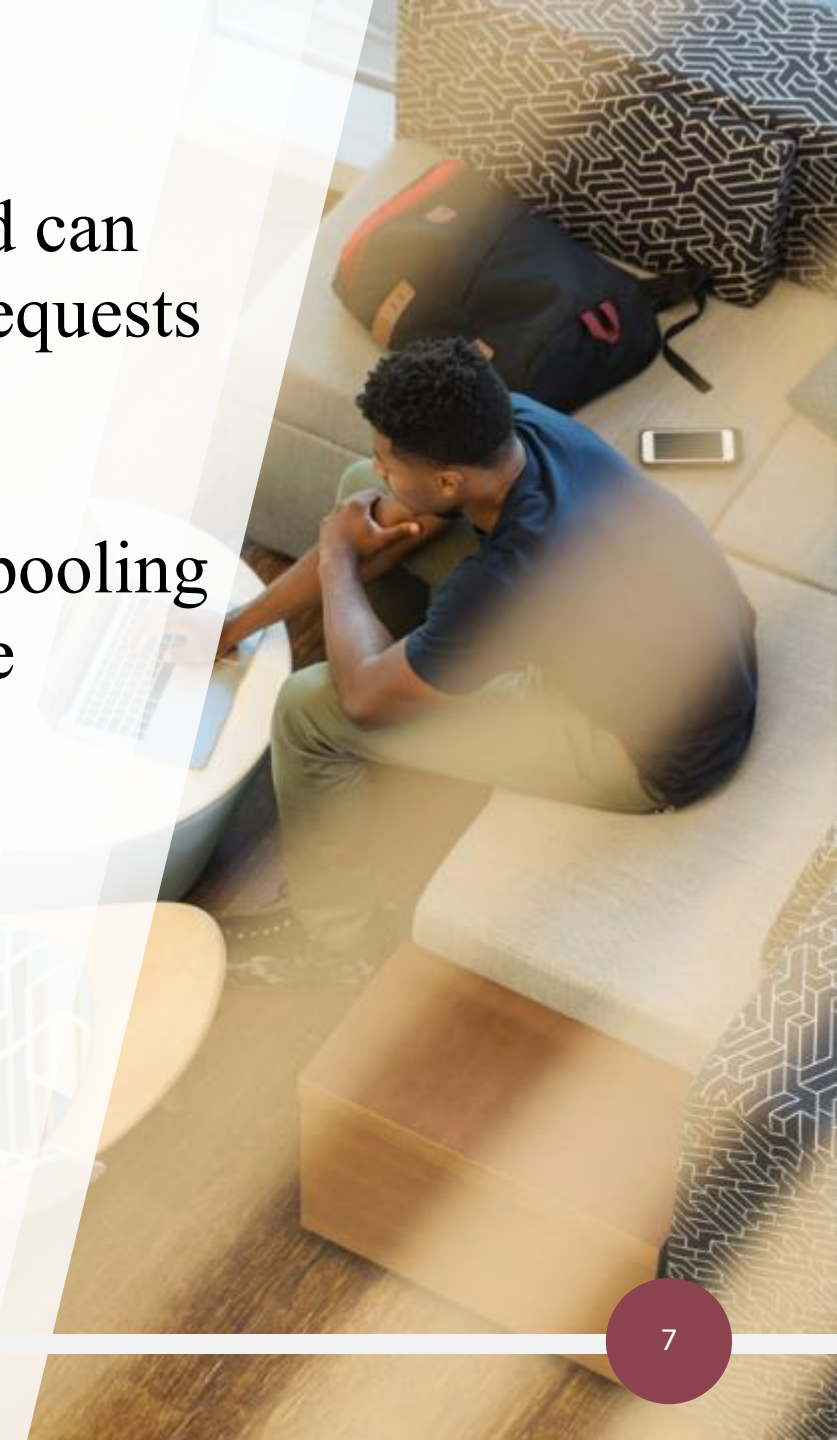


Virtual Threads - no need to pool them

- A virtual thread can run any code that a platform thread can run. This means that existing Java code that processes requests will easily run in a virtual thread.

- As platform threads are an expensive resource, thread pooling is attractive. However, as virtual threads are inexpensive resources, there is never any need to pool them.

- Therefore, rather than using a thread-pool based *ExecutorService*, use the virtual-thread-per-task *ExecutorService*.



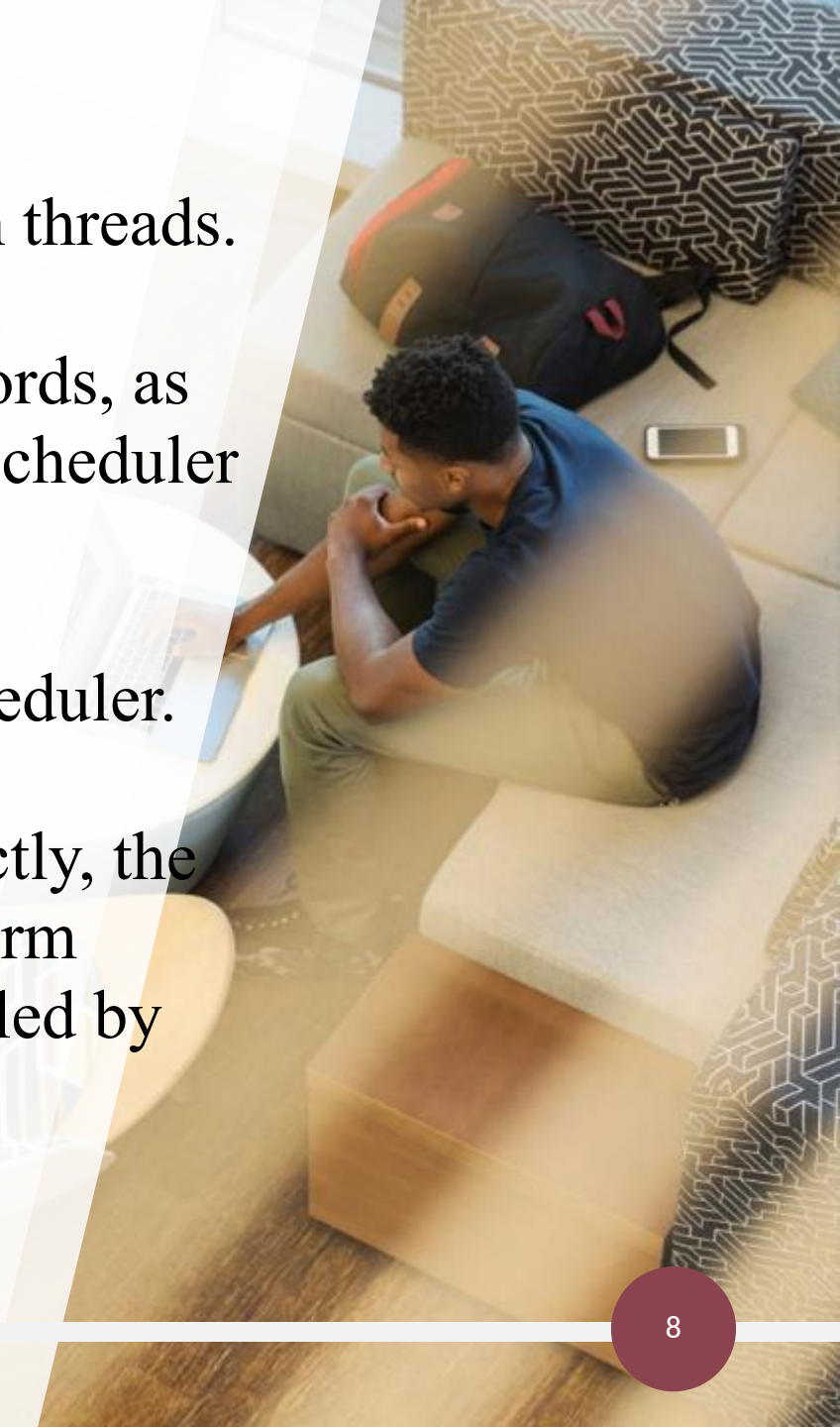
Virtual Threads - Scheduling

- Virtual threads are scheduled a little differently to platform threads.

- Platform threads use the OS scheduler directly; in other words, as they are implemented as OS threads, the JDK relies on the scheduler in the OS.

- Virtual threads, on the other hand, uses the JDK's own scheduler.

- So, rather than assigning virtual threads to processors directly, the JDK's scheduler assigns ("mounts") virtual threads to platform threads. These platform threads ("carriers") are then scheduled by the OS as usual.

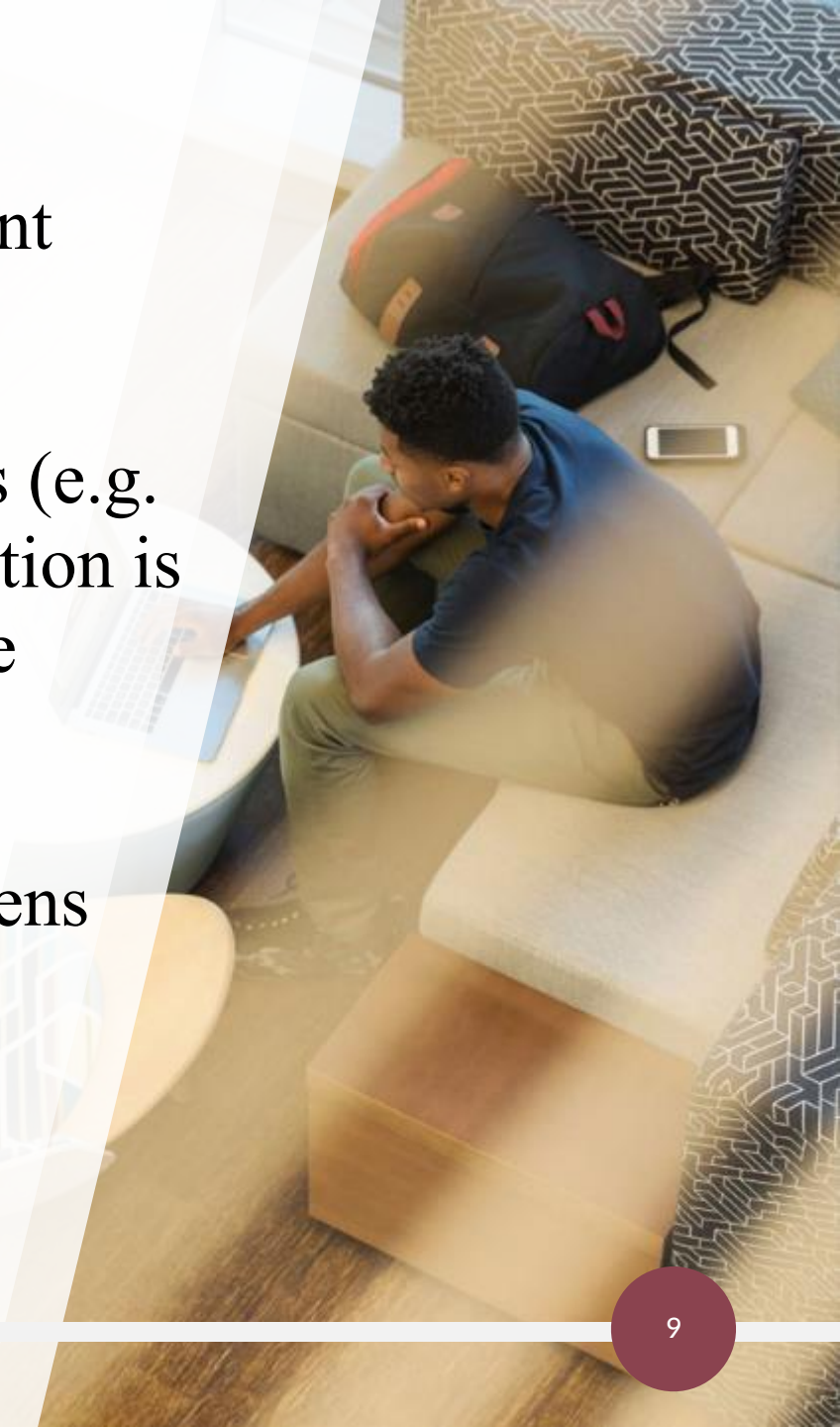


Virtual Threads - Scheduling

- A virtual thread can be mounted/unmounted on different carriers over the course of its lifetime.

- Typically, a virtual thread will unmount when it blocks (e.g. an I/O or database operation). When the blocking operation is complete, the virtual thread is mounted on any available carrier.

- The mounting and unmounting of virtual threads happens frequently and transparently, **without blocking** any OS threads.



	Platform Threads	Virtual Threads
Mapping to OS	1:1 mapping: each thread is an OS thread.	Many virtual threads are multiplexed over a smaller pool of OS threads.
Resource Overhead	Heavy (large stack sizes, higher memory consumption).	Lightweight (small stacks with minimal memory footprint).
Scheduling and Blocking	Managed by the OS; blocking occupies the OS thread.	Managed by the JVM; when blocked, the underlying OS thread is freed.
Scalability	Limited scalability—creating thousands can be impractical.	Highly scalable—can support tens of thousands to millions of threads.